

Programmeringsparadigmer 2025

Session 5: Rekursion

Problemer til løsning og diskussion

Hans Hüttel

7. oktober 2025

Opgaver, som vi helt sikkert vil tale om

1. (**10 minutter**) Funktionen `reverse` findes i Haskell-præludiet. Den vender en liste, således at f.eks. `reverse [1,2,3]` evaluere til `[3,2,1]`. Din opgave er nu at definere din egen version af denne funktion, `rev`. Prøv først at finde ud af, hvilken type `rev` skal have, og følg den generelle tilgang beskrevet i Afsnit 6.6.
2. (**15 minutter**) En liste $[a_1, a_2, \dots, a_n]$ er *aftagende*, hvis $a_1 \geq a_2 \geq \dots \geq a_n$. Skriv en funktion `descending`, der giver os `True`, når den liste, den får som argument, er aftagende, og `False` ellers. Eksempelvis skal `descending [6,5,5,1]` give `True`, og `descending ["plip", "pli", "ppp"]` skal give `False`. Hvad er dens type?
3. (**20 minutter**) Funktionen `isolate` tager en liste `l` og et element `x` og returnerer et par af to nye lister `(l1, l2)`. Den første liste `l1` indeholder alle elementer i `l`, der ikke er lig med `x`. Den anden liste `l2` indeholder alle forekomster af `x` i `l`.

- `isolate [4,5,4,6,7,4] 4` evaluere til `([5,6,7],[4,4,4])`.
- `isolate ['g','a','k','a'] 'a'` evaluere til `(['g','k'], ['a','a'])`.

Definer `isolate` i Haskell *uden* at bruge `fst`, `snd`, `head` eller `tail`. Hvad skal typen af `isolate` være?
Vink med vognstang: Placer det rekursive kald i en `let`- eller `where`-klausul og brug mønstre til at finde komponenterne i resultatet af dette kald.

4. (**20 minutter**) Funktionen `wrapup` er en funktion, der tager en liste og giver os en liste af lister. Hver liste i denne liste indeholder de på hinanden følgende elementer fra den oprindelige liste, der er identiske. For eksempel skal `wrapup [1,1,1,2,3,3,2]` give os listen `[[1,1,1],[2],[3,3],[2]]`, og `wrapup [True,True,False,False,False,True]` skal give os listen `[[True,True],[False,False,False],[True]]`. Definer `wrapup` i Haskell ved hjælp af rekursion¹, men *uden* at bruge `fst`, `snd`, `head` eller `tail`. **Vink med vognstang:** Husk definitionen af `isolate`-funktionen fra før.
5. (**15 minutter**) En tidligere videnskabs- og uddannelsesminister har besluttet at tage en universitetsuddannelse og forsøger nu at definere en Haskell-funktion `triples`, der tager en liste af tupler (hver tuple har præcis 3 elementer) og konverterer denne liste af tupler til en tuple af lister.

`triples [(1,2,3), (4, 5, 6), (7, 8, 9)]` skal producere `( [1,4,7], [2, 5, 8], [3, 6, 9] )`.

Ministeren skrev følgende stykke kode og en typespecifikation, men løb ind i problemer. Hvad er der mon i vejen?

```
triples :: Num a => [(a,a,a)] -> ([a],[a],[a])

triples [] = ()
triples [(a,b,c)] = ([a],[b],[c])
triples (x:xs,y:ys,z:zs) = [x,y,z] : Triples [(xs,ys,zs)]
```

Kan du gøre noget ved ministerens problemer? Hvordan kan Afsnit 6.6 hjælpe dig her?

Flere opgaver til løsning i dit eget tempo

Her er Afsnit 6.6 også nyttigt.

¹Ikke listeabstraktion – det var sidste uge!

- a) Funktionen `rle` er en funktion, der, når den får en liste `xs`, producerer en liste af par af elementer fra `xs` og heltal². Denne liste af par har sine elementer i den rækkefølge, de oprindeligt optrådte, og indeholder (x, n) , hvis der er n på hinanden følgende forekomster af x i listen. For eksempel skal

```
rle ['a','a','a','g','g','b','a','a']
```

give os listen `[('a',3),('g',2),('b',1),('a',2)]`, og

```
rle [1,1,1,2,2,1,3,3]
```

skal give os `[(1,3),(2,2),(1,1),(3,2)]`. Definer `rle` i Haskell. Prøv først at finde ud af, hvilken type `rle` skal have, og følg den generelle tilgang beskrevet i Afsnit 6.6.

- b) Definer en funktion `amy`, der fortæller os, om nogle elementer i en liste opfylder et givet prædikat. For eksempel, hvis

```
odd x = ((x `mod` 2) == 1)
```

så skal

```
amy odd [2,5,8,3,7,4]
```

returnere `True`, mens

```
amy odd [2,8,42]
```

skal returnere `False`.

- c) Lav en funktion `frequencies`, der, givet en streng `s`, opretter en liste af par `[(x1,f1),...,(xk,fk)]` således, at hvis tegnene `xi` forekommer et samlet antal af `fi` gange i listen `s`, så vil listen af par indeholde parret `(xi, fi)`. Som eksempel på dette skal

```
frequencies "regninger"
```

returnere listen

```
[('r',2),('e',2),('g',2),('n',2),('i',1)]
```

Find først ud af, hvilken type funktionen skal have.

- d) En sætning inden for talteori siger, at ethvert ikke-nul reelt tal x kan skrives som en *kædebrøk*. Det er et potentielt uendeligt udtryk på formen

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \frac{1}{a_4 + \frac{1}{\dots \frac{1}{a_n}}}}}} \quad (1)$$

For rationale tal vil a_i 'erne til sidst alle være 0, så kædebrøken er endelig; for irrationale tal vil den fortsatte brøk være uendelig. Se f.eks. [1] for mere. Målet med dette problem er at skrive en Haskell-funktion `cfrac`, der, givet et reelt tal r og et naturligt tal n , finder listen af de første n tal i kædebrøken for r . Hvad skal typen af `cfrac` være?

Litteratur

- [1] Wikipedia. Continued fractions. https://en.wikipedia.org/wiki/Continued_fraction.

²Denne funktion beregner det, der kaldes en run-length encoding, deraf navnet.